

COL1000: Lab 9

Semester 2, 2025–26

Instructions Upload your code to Gradescope as: `<entryNumber>_q<quesNumber>.py`. After uploading, run the autograder to check that all tests pass. Make sure not to include any custom prompt text in `input()` calls, and any extra text in `print()` statements. You must not use the `math`, `ast`, `intertool`, `sympy` libraries and advanced built-in python functions such as `split`, `splitlines`, `map`, `join`.

Question 1: Equal Pair Sums Write a program that reads a list A having **distinct** positive integers given on a single line (space-separated), and prints 1 if there exists distinct indices (i, j, k, l) such that $A[i] + A[j] = A[k] + A[l]$, and zero otherwise.

Input:

1 8 15 3 4 2

Output:

1

Note: A brute-force solution runs in n^4 steps. Devise a solution using a dictionary to reduce the number of steps to n^2 .

Question 2: Triplets in a list Write a program that reads a list A having **distinct** positive integers given on a single line (space-separated), and prints the total number of triplets $i < j < k$ such that

$$\delta(A[i]) = \delta(A[j]) = \delta(A[k]),$$

where $\delta(x)$ denotes the sum of digits in x .

Question 3: Primes Write an efficient program that reads an integer n and outputs the sorted list of all primes less than equal to n .

Example:

Input:

18

Output:

[2, 3, 5, 7, 11, 13, 17]

Hint: Use the fact that $n(\sum_{i=1}^n \frac{1}{i})$ is at most $4n \log n$.

Question 4: Permutation Guessing Game There is a hidden permutation P of the list $[1, 2, \dots, n]$, fixed in advance by the autograder and unknown to you.

Write a program that first reads n and then deduces P in at most n^2 attempts. Each guess must be a permutation of $[1, 2, \dots, n]$ printed as a list of length n , for example $[3, 1, 4, 2, 5, 6, 7, 8]$. After printing a guess, your program must immediately read a single integer from standard input (outputted by the autograder): this integer is the number of positions k such that your guessed list G satisfies $G[k] = P[k]$ (i.e., the number of elements in the correct position).

The code terminates on correctly guessing the permutation (i.e. when the autograder certifies that all n positions are matching) or on reaching the limit of n^2 -guesses.

Example:

Suppose the hidden permutation (fixed by autograder) is: $[8, 7, 1, 2, 3, 4, 5, 6]$. Then following is a valid sequence of input and output.

```
8
[1, 2, 3, 4, 5, 6, 7, 8]
0
[8, 2, 3, 7, 1, 4, 5, 6]
4
[8, 7, 1, 2, 3, 4, 5, 6]
8
```

Hint: You can solve the problem in n^2 guesses, even if consecutive permutation guesses differ in at most two indices. You can attempt the same manually here.